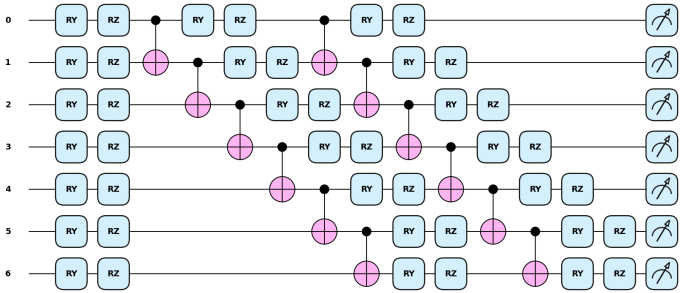
OHIO UNIVERSITY

Variational Quantum Eigensolver and Near Quantum Advantage

Presented by: **Joshua M Maldonado**

jm998521@ohio.edu

OHIO UNIVERSITY



Outline

- How do Eigensolvers work?
  - Example with Toy Problem
- What is a Variational Quantum Eigensolver (VQE)?
  - Example with Toy Problem
  - Advantages and Disadvantages
- How close are we to ”Quantum Advantage”

OHIO UNIVERSITY

How do Eigensolvers Work?

- The classic eigenvalue problem:

$$\hat{H}\,|\psi\rangle = E\,|\psi\rangle$$

- Several ways to solve, the "easiest" for a person: $\det(H - \lambda\,\mathbb{I}) = 0$
  - This is *computationally horrible* for large matrices, and very quickly leads to numerical instability
- Numerically, computers employ other methods based on whether the matrix is dense or sparse and Hermitian or not:
  - Iterative QR algorithm, for dense matrices [1]
  - Krylov subspace projection methods for sparse, Hermitian matrices [2]

OHIO UNIVERSITY

Eigensolver Example

- Example of 1D harmonic oscillator:

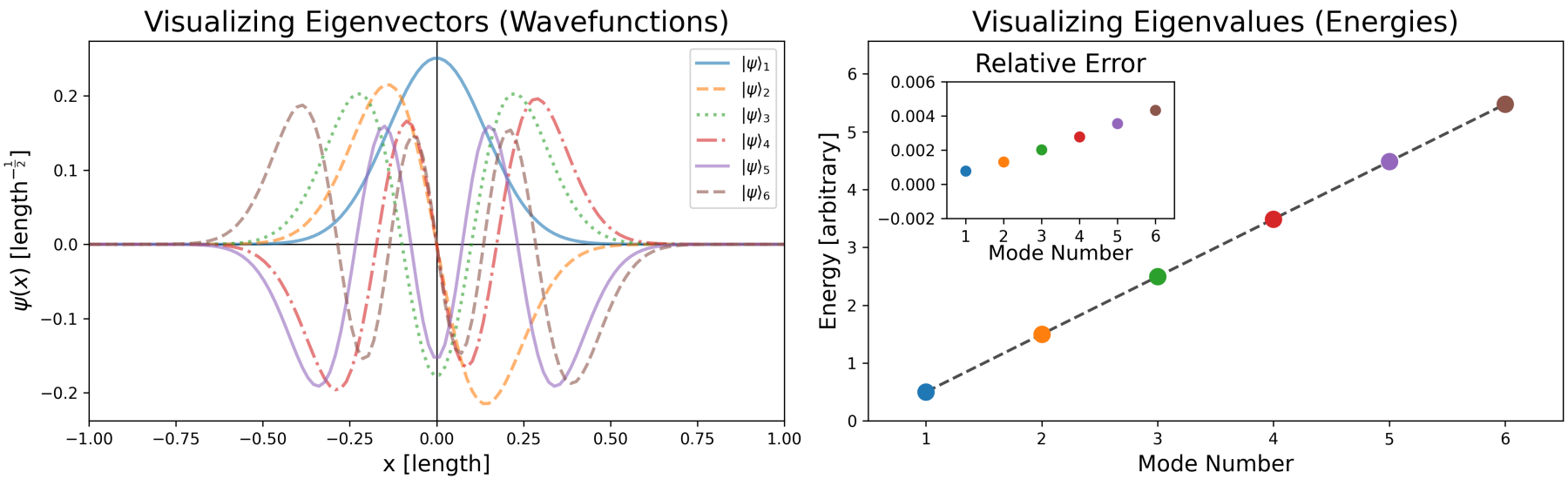
$$\hat{H} = \hat{T} + \hat{V}(x) = -\frac{1}{2\,m}\frac{\partial^2}{\partial x^2} + \frac{1}{2} m\,\omega^2 x^2$$

$$\hat{H}\,|\psi\rangle = E\,|\psi\rangle$$

```python
import numpy as np
from scipy.sparse import diags


###   ###   ###   constants   ###   ###   ###
hbar = c = 1  # as god intended
mass = 50  # example
omega = 1  # example

n = 128  # number of elements (2^7 = 128, nice for qubits later)


###   ###   ###   handy functions   ###   ###   ###
def harmonic_oscillator_potential(x,
                                  omega=omega,
                                  m=mass):
    V_vector = 0.5 * m * omega ** 2 * x ** 2  # calculate the potential
    V_matrix = diags(V_vector).toarray()  # operator-ify that vector
    return V_matrix

###   ###   ###   the actual code   ###   ###   ###
x, dx = np.linspace(-1, 1, n, retstep=True)  # real-space (it's where it's at!)
# finite differencing 2nd derivative (good enough for toy problem):
d2_by_dx2 = diags([1., -2., 1.], [-1, 0, 1], shape=(n, n)).toarray() / (dx ** 2)

T = - (1 / (2 * mass)) * d2_by_dx2    # kinetic energy
V = harmonic_oscillator_potential(x)  # potential energy
H = T + V  # define hamiltonian

# H psi = E psi  <- omg an eigenvalue problem!
eigenvalues, eigenvectors = np.linalg.eigh(H)
```

Eigensolver Code Output (first 6 modes)

Solutions to $\hat{H}|\psi\rangle = E|\psi\rangle$



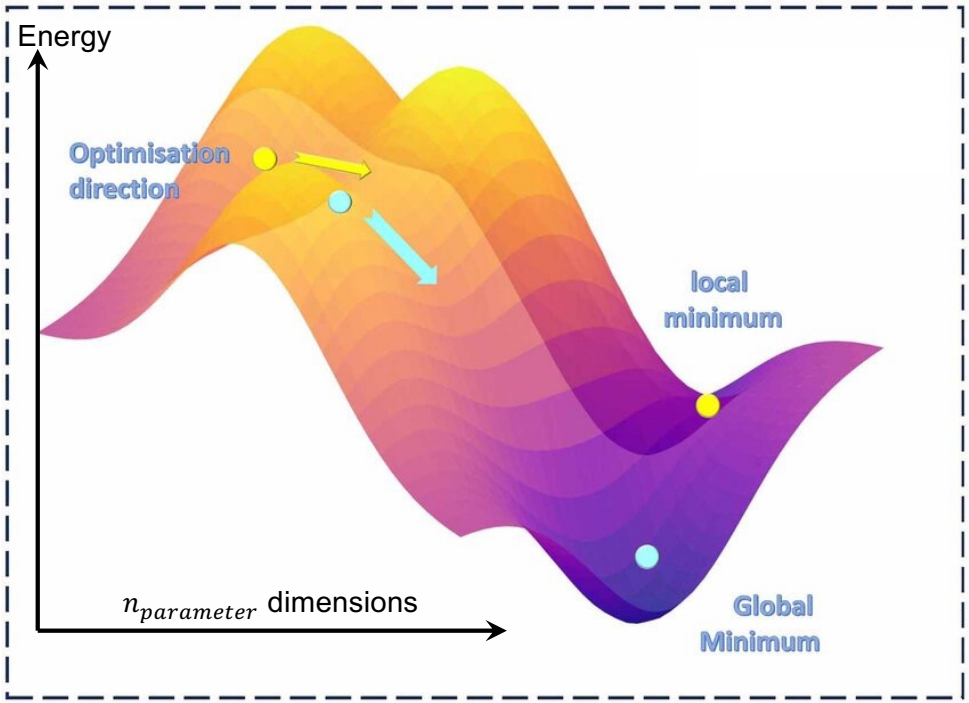
OHIO UNIVERSITY

What is a Variational Quantum Eigensolver (VQE)?

- "Variational" meaning "start with educated guess" and iteratively refine
- **Needs four things:**
  1. An operator that you care about, $\hat{H}$
  2. An initial guess with a *parameterized* wavefunction, $|\psi(\vec{\theta}_{\text{guess}})\rangle$
     - Each parameter is associated with a quantum gate
  3. An estimator for $\mathrm{E} \approx \langle\psi(\vec{\theta})|\, \hat{H}\, |\psi(\vec{\theta})\rangle$
  4. An optimizer to minimize the energy, E
- Steps (3 and) 4 means that this is a *hybrid* method, requiring both classical and quantum computations

3. An estimator for $\mathrm{E} \approx \langle \psi(\vec{\theta}) | \hat{H} | \psi(\vec{\theta}) \rangle$
4. An optimizer to minimize the energy, E

What is a Variational Quantum Eigensolver?



Energy landscape

Figure adapted from [3]

OHIO UNIVERSITY

What is a Variational Quantum Eigensolver?

- What is this method doing?
  - Varying the parameters, $\vec{\theta}$, to minimize the resulting energy
  - Some number of iterations later, we have **hopefully** converged to our minimum eigenvalue E, *the ground state*
- We now have a computational problem with different scaling!
  - **Classical Eigensolver**: $O(n^3)^*$ in time and $O(n^2)^*$ in space complexity
  - **VQE:** Time complexity based on estimator and optimizer, but this has linear, $\boldsymbol{O(n)}$ **space complexity!** [4]
    - We've even recently improved measurements needed from $O(n^4)$ to $O(n^3)$ [5] !

*can be improved a bit for sparse matrices

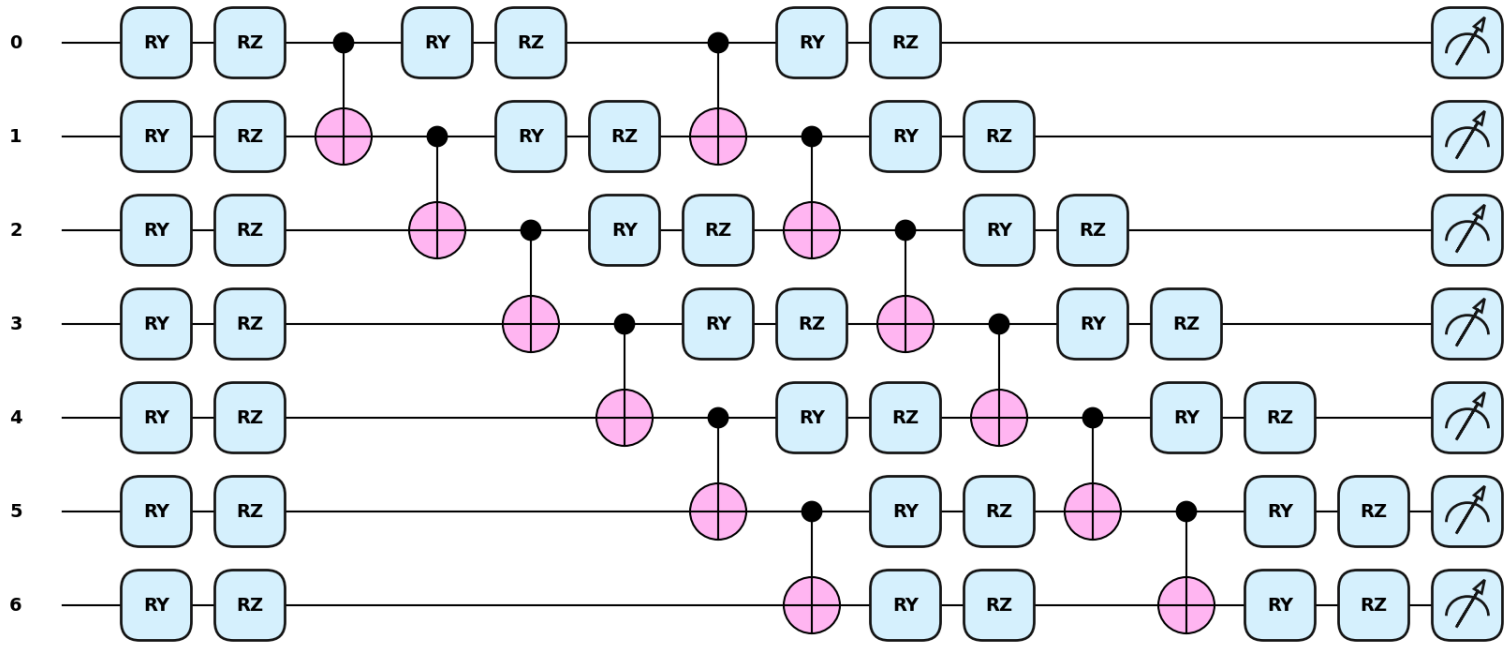
OHIO UNIVERSITY

$$\hat{H} = \hat{T} + \hat{V}(x) = -\frac{1}{2\,m}\frac{\partial^2}{\partial x^2} + \frac{1}{2} m\,\omega^2 x^2$$

VQE Example (kind of)

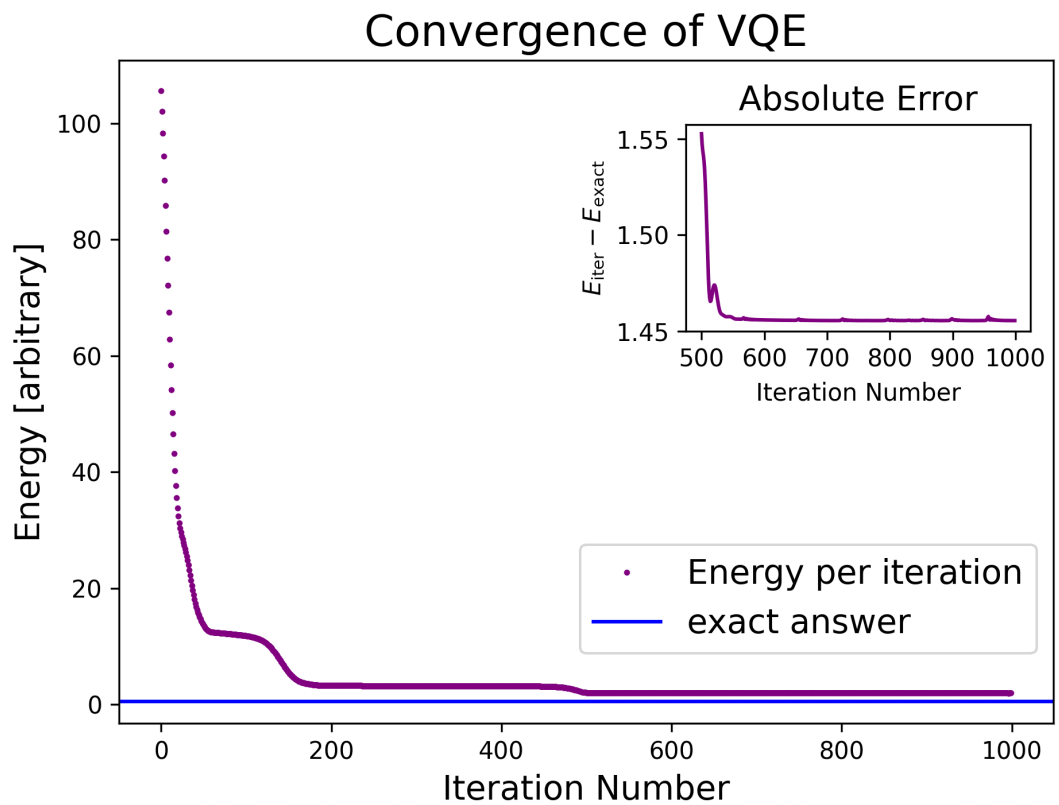
- Due to length of code, I'll just describe the steps (code available at https://github.com/Ub3rJosh/quantum_computing_course)
  1. The usual code setup. Notably, I keep the same `m`, `omega`, `x`, and `n` the same as the classical example
  2. Next, we turn to expressing $\hat{H}$ on (simulated) quantum hardware
     a) Need the number of qubits: $\log_2 128 = 7$ qubits
     b) Need to express in terms of pauli matrices: 149 different terms
        - This means that I have 149 *measurements* per optimization step
     c) Look at the dimensionality of parameter space for wavefunction, $|\psi(\theta)\rangle$
        - (parameterized "layers") x (rotations per qubits) x (qubit) = $3 \cdot 2 \cdot 7 = 42$ parameters to define $|\psi\rangle$ (explained more on next slide)
  3. Iterations!
     a) Perform all 149 measurements per iteration
     b) Estimate energy and slide "down" in the energy landscape, tuning the parameters as we go
     c) go to (a) `max_iter` times

VQE Example Circuit (thanks pennylane)



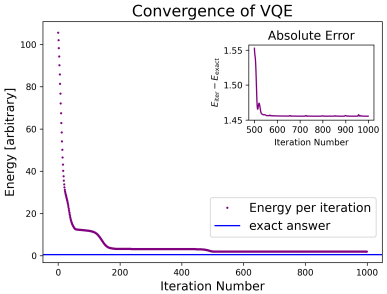
- (parameterized “layers”) x (rotations per qubits) x (qubit) = $3 \cdot 2 \cdot 7 = 42$ parameters to define $|\psi\rangle$

VQE Example Result



OHIO UNIVERSITY

VQE Example … What Happened?



- Did my example work? No, not really. Was this user error? Probably.
- Challenges with VQE:
  1. Initial guess in parameters
  2. Choice in optimizer, number of iterations, and step size in energy landscape
  3. ”Barren plateaus” in the energy landscape
  4. Noise from quantum tomfoolery! (None in this example.)
- In the real world:
  1. There are many ”different” VQEs:
     a) VQE for not just the ground state
     b) Adaptive VQE
     c) And many more…

OHIO UNIVERSITY

What is a Quantum Variational Eigensolver?

Advantages

- Better time and space scaling, making **very** large matrices feasible!
  - **$O(n)$ space complexity**
- Since most current problems have at least some information known about them, one can likely make a reasonable guess

Disadvantages

- You need a "good" initial guess, and need "enough" iterations for convergence
- Matrix must be expressed as a sum of Pauli matrices: $\{\sigma_x, \sigma_y, \sigma_z, \mathbb{I}\}$
- All the usual stuff with quantum computers
  - ...*and* you need a classical computer and quantum computer to talk to each other!
- All the usual stuff for optimizers

OHIO UNIVERSITY

How close are we to “Quantum Advantage”?

- Short answer: not very
- Longer answer:
  - VQE allows for very large Hamiltonians to be considered, where classical Eigensolvers would no longer be feasible
  - Need the quantum hardware for this to be done:
    - In principle, this can be done classically, but won’t benefit from space complexity scaling. So, very large matrices cannot be considered
  - Issues with noise in quantum systems:
    - Requires fault tolerant qubits and, in particular, **fault tolerant gates**

Since VQE has particular applications with **very** large matrices, its ”quantum advantage” requires considerable scaling in quantum hardware (on top of its previously mentioned disadvantages).

?

OHIO UNIVERSITY

References

[1] https://blogs.mathworks.com/cleve/2019/08/05/the-qr-algorithm-computes-eigenvalues-and-singular-values/

[2] https://phys.au.dk/~fedorov/Numeric/13/krylov.pdf

[3] Zhao, Runsheng & Cheng, Tao & Wang, Rui & Fan, Xingkui & Ma, Hongyang. (2024). Artificial intelligence warm-start approach: optimizing the generalization capability of QAOA in complex energy landscapes. New Journal of Physics. 26. 10.1088/1367-2630/ad4629.

[4] Shkolnikov, Vlad O., et al. "Avoiding symmetry roadblocks and minimizing the measurement overhead of adaptive variational quantum eigensolvers." Quantum 7 (2023): 1040.

[5] Jules Tilly, Hongxiang Chen, Shuxiang Cao, Dario Picozzi, Kanav Setia, Ying Li, Edward Grant, Leonard Wossnig, Ivan Rungger, George H. Booth, Jonathan Tennyson,The Variational Quantum Eigensolver: A review of methods and best practices,Physics Reports,Volume 986, 2022

Fun fact

- For a given square matrix H, $\quad H = \psi \cdot E \cdot \psi^{\dagger}$

```
print(f"H = psi @ e @ psi^dagger  max error = {np.max(H - (eigenvectors @ diags(eigenvalues) @ eigenvectors.transpose().conjugate())):.4}")
```

✓ 0.0s

```
H = psi @ e @ psi^dagger  max error = 1.705e-13
```

OHIO UNIVERSITY

```python
import numpy as np
from scipy.sparse import diags


###   ###   ###   constants   ###   ###   ###
hbar = c = 1  # as god intended
mass = 2   # example
omega = 1  # example

n = 1000  # number of elements (*large* for nice plots)


###   ###   ###   handy functions   ###   ###   ###
def harmonic_oscillator_potential(x,
                                  omega=omega,
                                  m=mass):
    V_vector = 0.5 * m * omega ** 2 * x ** 2  # calculate the potential
    V_matrix = diags(V_vector)  # operator-ify that vector
    return V_matrix

def normalize(psi):
    psi_dagger = psi.transpose().conjugate()  # as the name implies
    normalized_psi = psi / np.sqrt(np.abs(psi_dagger @ psi))  # psi / sqrt || <psi|psi> ||
    return normalized_psi


###   ###   ###   the actual code   ###   ###   ###
x, dx = np.linspace(-5, 5, n, retstep=True)  # real-space (it's where it's at!)
d2_by_dx2 = diags([1., -2., 1.], [-1, 0, 1], shape=(n, n)).toarray() / (dx ** 2)  # finite differencing 2nd detivative

T = - (1 / (2 * mass)) * d2_by_dx2    # kenetic energy
V = harmonic_oscillator_potential(x)  # optential energy
H = T + V  # define hamiltonian

# H psi = E psi  <- omg an eigenvalue problem!
eigenvalues, eigenvectors = np.linalg.eigh(H)
```

-1

OHIO UNIVERSITY

```python
import numpy as np
from scipy.sparse import diags
from scipy.linalg import eigh
import pennylane as qml
from pennylane import numpy as pnp
from tqdm import tqdm

###   ###   ###   constants   ###   ###   ###
hbar = c = 1  # as god intended
mass = 50  # example
omega = 1  # example

n = 128  # number of elements, same as before

###   ###   ###   handy functions   ###   ###   ###
def harmonic_oscillator_potential(x,
                                  omega=omega,
                                  m=mass):
    V_vector = 0.5 * m * omega ** 2 * x ** 2  # calculate the potential
    V_matrix = diags(V_vector).toarray()  # operator-ify that vector
    return V_matrix


###   ###   ###   more problem setup   ###   ###   ###
x, dx = np.linspace(-1, 1, n, retstep=True)  # real-space (it's where it's at!)
# finite differencing 2nd derivative (good enough for toy problem):
d2 = diags([1., -2., 1.], [-1, 0, 1], shape=(n, n)).toarray() / (dx**2)
T = -0.5 / mass * d2
V = harmonic_oscillator_potential(x)
H_matrix = T + V

###   ###   ### Convert the Hamiltonian matrix to a Pauli operators   ###   ###   ###
n_qubits = int(np.log2(n))    # log2(128) = 7 for this example
print(f"Converting {n}x{n} Hamiltonian to Pauli operators ({n_qubits} qubits)...")
# The wire_order argument specifies the qubit labels for the matrix
hamiltonian = qml.pauli.pauli_decompose(H_matrix, wire_order=range(n_qubits))
print(f"Number of Pauli terms: {len(hamiltonian.ops)}")


###   ###   ###   Define the ansatz (variational circuit) for 7 qubits   ###   ###   ###
dev = qml.device('default.qubit', wires=n_qubits)

# Choose a simple hardware-efficient ansatz with 2 repetitions
def ansatz(params):
    # Initialize all qubits in |0> (no Hartree-Fock state needed for this problem)
    # Apply layers of single-qubit rotations and CNOTs
    for layer in range(2):
        for i in range(n_qubits):
            qml.RY(params[layer, i, 0], wires=i)
            qml.RZ(params[layer, i, 1], wires=i)
        for i in range(n_qubits - 1):
            qml.CNOT(wires=[i, i+1])
    # Optionally add a final layer of rotations
    for i in range(n_qubits):
        qml.RY(params[2, i, 0], wires=i)
        qml.RZ(params[2, i, 1], wires=i)

@qml.qnode(dev)
def cost_fn(params):
    ansatz(params)
    return qml.expval(hamiltonian)


###   ###   ###   Set up the optimization   ###   ###   ###
# Number of parameters: (2 layers * 2 rotations per qubit) + (final layer * 2) = 3 layers * 7 qubits * 2 = 42
np.random.seed(17)
# init_params = pnp.random.random((3, n_qubits, 2), requires_grad=True)  # random start
init_params = pnp.zeros((3, n_qubits, 2), requires_grad=True)  # start from all zeros

step_size = 0.03
# optimizer = qml.GradientDescentOptimizer(stepsize=step_size)  # this gave bad convergence
optimizer = qml.AdamOptimizer(stepsize=step_size)  # this gave "better" convergence

print("\nStarting VQE optimization...")
max_iter = 1000
iter_energies = []
params = init_params
for i in tqdm(range(max_iter)):  # this loop takes a couple minutes
    params, energy = optimizer.step_and_cost(cost_fn, params)
    iter_energies.append(energy)

###   ###   ###   Compare with exact result   ###   ###   ###
exact_energy = 0.5
print("\n--- Final Results ---")
print(f"Exact ground state energy (known result): {exact_energy:.8f}")
print(f"VQE estimated ground state energy: {energy:.8f}")
print(f"Error: {abs(energy - exact_energy):.2e}")
```